1-2 Loop Engineering：积木、文件与工具

> [!TIP] 阅读指南
>
> 上一节我们讲了为什么需要 Loop Engineering——让 AI 从“等指令的执行者”变成“自己找活干的成员”。
>
> 这一节我们来拆解它的具体构造：六个积木块怎么组合、一个 Loop 怎么运转。

2.1 六个积木

> [!TIP] 来源说明
>
> 以下“六个积木”框架，是我综合了以下来源之后归纳出来的 LE 框架要素
>
> - Cobus Greyling 的 `loop-engineering` 仓库
> - Addy Osmani 的博客
> - Codex CLI 的相关实践

Loop Engineering 把“Agent 怎么自动工作”拆成了六个可以组合的积木块。理解了这六个积木，你就理解了整个体系。

积木一：调度（Scheduling / Automation）

类比： 手机闹钟。

做什么： 让 Loop 在指定时间自动启动。比如每天早上 9 点跑一次，或者每次有人提 PR 的时候跑一次。

用什么实现： GitHub Actions 的定时触发（cron），或者系统级的 cron / systemd timer。

没有调度，Loop 就不是 Loop——它只是一次性的任务。调度是让 Loop 真正“循环”起来的那个心跳。

积木二：技能（Skills）

技能的作用很明确：给 Agent 一份持久的“知识文件”，告诉它这个项目的规则、约定、怎么做某件事。Skill 保存在文件里，Agent 每次干活前先读 Skill，不会忘记。

举个例子： 你写一个 `skills/loop-triage.md`，告诉 Agent “issue 带 bug 标签的优先级最高，feature request 先放到讨论区”。Agent 每次处理 issue 都会遵守这个指南。

Skills 也是把“意图”从一次性的对话中解放出来的关键。我在第 10 章详细讲过 Skill 的原理，这里的用法异曲同工。

这里顺便提一下，上个小节，我们演示了一段 Prompt，看起来和这里的 Skill 功能类似。那么这里为什么又换成了 Skill?

如果你有这个疑问，说明你对 Agent 的体系还没有理解的很清楚。

Prompt 更多的是一次性的，他不具备复用性。适合执行一些临时的、零散的任务。如果是需要长期维护、打磨的任务，那么 Skill 更合适。

如果你看过我的 Hermes 课就会知道，Prompt 大部分的时候会沉淀为 Skill，从临时性的提示词变成知识资产。

积木三：隔离工作区（Worktrees）

类比： 给实习生一个“临时工位”，干砸了不影响正式办公室。

做什么： 每次 Agent 尝试修改代码，先创建一个独立的 git worktree（相当于临时分支加独立目录）。在这个隔离区里随便改。改好了合回去；改砸了直接扔掉，不动主线。

为什么需要这个？ 一旦你跑多个 Agent 并行工作，文件冲突就会成为最大的失败原因。两个 Agent 同时写同一个文件，和两个工程师同时改同一行代码没有区别。Worktree 从根本上解决了这个问题——每个 Agent 有自己的工作副本，互不干扰。

举个例子

Agent 尝试修一个 bug：

创建 worktree → 在 worktree 里改代码 → 跑测试 → 测试过了合回 main → 测试没过直接删 worktree，main 不受影响。

至于怎么创建 worktree，我觉得看到这里应该不是什么问题了——如果你不会，直接告诉你的 Agent，让他来创建。

积木四：接口——让 Agent 有手有脚

类比： 给实习生配了电脑，但没装任何软件。他只能干坐着。

做什么： Agent 要操作一个外部平台（GitHub、数据库、Slack、飞书），前提是那个平台提供了可编程的接口。没有接口，Agent 就什么也做不了——它没有手也没有脚。

什么接口？ 三种方式，按推荐顺序：

- **CLI（推荐）：** 命令行工具，Agent 可以直接调用。公共平台通常有官方的 CLI（GitHub CLI、Vercel CLI、AWS CLI 等），自己公司内网的服务也可以封装成 CLI。CLI 的详细设计可以参考我的 Hermes 课程中关于 CLI 封装的内容。
- **MCP：** 通过 Model Context Protocol 连接，我们在第 7 章详细讲过。MCP 的好处是 Agent 可以直接理解工具的描述和参数，不需要额外解释。

- **RESTful API：** 最传统的方式，Agent 直接发 HTTP 请求。但相比 CLI 和 MCP，Agent 需要自己处理认证、拼接 URL、解析响应，出错概率更高。

一个简单的判断标准：如果某个平台没有开放 CLI 或 API，Agent 就碰不到它。你给 Agent 配的接口越多，它能做的事就越多——接口就是它的手和脚。

关于接口这个话题——为什么它是企业 Agent 化最大的瓶颈，没有接口怎么办、为什么推荐 CLI 而不是 MCP 或 RESTful API——我们在第三节单独展开讨论。

积木五：子智能体（Sub-agents）

类比： 一个干活（Maker），一个检查（Checker）。不能让同一个人又写代码又自己检查——那等于没检查。

做什么： 把一个 Loop 里的工作拆成两个 Agent 完成。Implementer Agent 负责写代码，Verifier Agent 负责跑测试、检查结果。写代码的和检查的不是同一个人，避免“自欺欺人”。

为什么这一点在 Loop 里特别重要？ 因为 Loop 是在你不在场的时候运行的。如果只有一个 Agent 既写又查，它对自己的代码总是太宽容了。一个独立的 Verifier 是你能放心走开的唯一理由。

积木六：记忆/状态（Memory / State）

类比： 贴在墙上的工作日志——谁看了都知道现在是什么状态、上次干了什么。

做什么： 让 Agent 的状态不依赖对话上下文。上下文会被清空，但文件不会。状态存在文件里——`STATE.md` 记录当前状态，`loop-run-log.md` 记录每次运行的日志。Agent 每次醒来先读这两个文件，就知道“我是谁、我在哪、我要干什么”。

为什么这很重要？ 我在第 11 章讲 Agent 记忆系统时说过，模型会忘记一切。但仓库不会。把记忆放在磁盘上，而不是上下文里，是每一个长期运行的 Agent 都依赖的核心技巧。

举个例子： `STATE.md` 记录“上次运行时间：2026-07-26 08:47，高优先级待办：无”。Agent 早上 9 点被闹钟叫醒，先看 `STATE.md`，知道一切正常，跳过，等下一轮。

强烈建议，这里的设计参考我们之前讲解的 LangGraph 的设计。

一个 Loop 究竟怎么运转

把六个积木拼起来，一个标准 Loop 的流程是这样的：

第 1 步 Schedule（闹钟响了）

定时触发 Loop

第 2 步 Triage（看工作日志）

Agent 读取 LOOP.md + STATE.md，

确定“我今天该干什么”

第 3 步 Worktree（开个临时工位）

创建隔离的 git worktree，随便改

第 4 步 Implement + Verify（干活 + 检查）

Implementer Agent 动手改代码

Verifier Agent 跑测试 + gate 检查

第 5 步 Gate Check（安全检查）

loop-gate 机械检查：这次改动有没有碰

.env、secrets、migrations 等禁区？

第 6 步 Human Gate（危险操作送来给你看）

安全操作（.md 文件）→ 自动合

危险操作（碰到禁区）→ 暂停，等人拍板

第 7 步 Update State（写工作日志）

更新 STATE.md + loop-run-log.md

汇报干了什么，花了多少 token

第 8 步 回到第 1 步，等下一轮闹钟

注意第 5 步和第 6 步的区别。第 5 步是**机械检查**——由 `loop-gate` 工具硬性对照 `gate.yaml` 文件，不允许任何 Agent 绕过。第 6 步是**人工判断**——安全操作自动执行，危险操作暂停等待你拍板。这两层合在一起，既保证了速度，又守住了底线。

2.2 三个核心文件

Cobus Greyling 在 `loop-engineering` 仓库中定义了三个核心文件作为参考。需要说明的是，这不是官方标准——每个人都可以根据自己的项目需求设计文件结构，这三个文件只是目前社区里最常用的实践。

LOOP.md —— “岗位职责说明书”

这是整个 Loop 的定义文件。它写了：

- 这个项目有哪些 Loop（Daily Triage、PR Babysitter、Dependency Sweeper 等）
- 每个 Loop 的岗位职责是什么、用什么 skill、什么节奏、自动化到什么程度
- 多个 Loop 之间的优先级（哪个先跑哪个后跑）

Agent 每次都从看 `LOOP.md` 开始。`LOOP.md` 自己也在 git 里，可以版本管理。

STATE.md —— “工作日志”

这是 Loop 自动维护的状态文件。它写了：

- 上次运行时间
- 当前有什么高优先级的事要做
- 有什么正在观察的事（还没决定要不要做）
- 有什么低优先级的“噪音”（先不管）

Agent 每次都更新这个文件。人类每周看一次，决定下一步方向。

gate.yaml —— “禁区地图”

这是机器可读的安全规则。它写了：

- 绝对不许碰的路径（denylist）：`.env`、`/secrets/`、`auth/**`、`payments/**` 等
- 可以自动合并的路径（autoMergeAllowlist）：`.md` 文件、测试文件

`loop-gate check` 命令会在每次 Agent 想改代码时，机械地对照这个文件。这保证了对安全规则的强制执行——不是靠 Agent 自觉，是靠程序硬挡。

2.3 配套工具：Cobus Greyling 的 CLI 实现

Loop Engineering 目前没有所谓的官方标准。事实上，包括 Skill 在内，都不是什么标准，现在的 AI 发展还处于草莽时代，这和传统的 Web W3C 不一样。每个人都可以根据自己的需求来构建工具链。

作为参考，Cobus Greyling 在 `loop-engineering` 仓库中提供了一套完整的 CLI 工具集，每个工具都可以用 `npx` 一行命令调用。下面介绍这套实现，方便你理解一个完整的工具链大概长什么样：

工具	干什么	一行示例
loop	统一入口，包含 init、doctor、status、audit	npx @cobusgreyling/loop doctor .
loop-init	搭骨架——生成 LOOP.md/ STATE.md/ gate.yaml	npx @cobusgreyling/loop-init . --pattern daily-triage
loop-audit	打分——评估项目离“Agent 能自动管理”还差多远（0-100 分）	npx @cobusgreyling/loop-audit . --suggest
loop-cost	算钱——预估这个 Loop 跑一次要消耗多少 token	npx @cobusgreyling/loop-cost --pattern daily-triage
loop-sync	对账——检查 STATE.md 和 LOOP.md 有没有说的一致	npx @cobusgreyling/loop-sync .
loop-context	长期记忆——Agent 跑太久了，帮它记住上下文，防止忘记	npx @cobusgreyling/loop-context --check
loop-gate	安检——机械检查“这次改动有没有踩禁区”	npx @cobusgreyling/loop-gate check --action auto-merge
loop-worktree	隔离工位——创建/销毁 git worktree	npx @cobusgreyling/loop-worktree create --run-id bugfix-01

这套工具的设计思路很清晰：**每一步都有对应的工具，每个工具只做一件事。** 它们可以独立使用，也可以组合成完整的自动化流水线。当然，这只是 Cobus 个人的实现思路——你可以照搬，也可以只借鉴其中一部分，或者完全自己从头写一套。

> [!TIP]
>
> 你可能觉得这个仓库的 Stars 不多，这很正常。因为类似 LE 这种机制，他很难有一套通用的标准，所以每个人都会根据自己的具体情况来实现。所以上面给出这个仓库的参考，也是方便大家理解 LE，而不是照抄。

2.4 Loop Engineering 和 /goal 的区别

我在 Hermes 课程里讲解过 `/goal` 模式是怎么使用的。那么一个很自然的问题，LE 和 `/goal` 有什么区别吗？看起来都像是让 Agent 自己完成任务。

区别非常大。`/goal` 的目的是为了让 Agent 完成一个任务。但是 Loop 的目的是让 Agent 接管你某个方向的工作。他们的切面不同，你可以在 Loop 里使用 `/goal` 模式。

Codex CLI 和 Hermes 都有 `/goal` 命令（Hermes 在 v0.13.0 引入，称作“Ralph loop”，直接受 Codex CLI 的启发）。很多人把 Loop Engineering 和 `/goal` 搞混，因为它们都涉及“Agent 自动工作”。但它们的本质不同。

	Loop Engineering	/goal（Codex CLI / Hermes）
触发方式	按时间/事件自动触发（闹钟模式）	你手动输入 /goal 我要重构登录模块，人工触发
谁发现问题	Agent 自己发现（发现问题就是它的工作）	你发现问题，然后告诉 Agent
什么时候结束	不结束——它是个循环，一轮完了等下一轮	有终点——目标达成了就结束
适合做什么	持续性的运维工作：每天分 issue、每周更新依赖	一次性任务：写一个新功能、重构一个模块
你扮演的角色	监督员——看日志、处理异常	发起人——你提需求，Agent 干到完成为止
举例	“每天早上帮我分 GitHub issue”	“帮我把登录模块从 Express 重构到 Fastify”

2.5 三阶段升级路线

LE 是一项高自由的模式，这意味着它必然伴随着高风险。他只适合有丰富开发和运维经验的人。新手还是老老实实的逐步提升，不要一开始就追求全自动或者半自动。

建议分阶段逐步提升。

第一阶段：L1 —— 只看不动

Loop 每天跑一次，输出报告到 `STATE.md`。Agent 只分析问题（“今天有 3 个新 issue，其中 1 个像 bug”），不自动做任何操作。人类每周审查 `STATE.md`，决定下一步。

适用：刚接入 Loop 的项目，还没建立信任。

第二阶段：L2 —— 自动做低风险的事

低风险操作（补丁升级、issue 分类、markdown 文件编辑）自动执行。高风险操作（数据库迁移、支付模块改动）暂停等人确认。

适用：Loop Ready Score 60+，信任度中等。

第三阶段：L3 —— 高自主

大部分操作自动完成。人类只看异常报告。Loop Ready Score 80+。

适用：成熟项目，已跑 L2 一个月以上无事故。

2.6 如何开始自己的 Loop Engineering?

如果看到这里，你还需要我来为你写一个示例，这可能说明你还是没有适应 Agent 时代。在这里我几乎无法给你一个可以运行的 LE。

实际上，我认为你应该先设想一个最简单的场景：修改某份文档。以我的一个工作场景为例子，我的《绿皮书》教材里可能会存在一些错别字或者过时的内容。而当我发现这些错别字时，或者读者提出了比较好的建议，我会把这些建议直接推送到 github 上。

这就是一个最小的 LE 案例——让你的 LE 去解决这些 issue。反复打磨这个案例，一直到他能稳定的运行。然后再进一步的升级到 L2。

此外，不建议直接使用上文提到的仓库，而是在理解这个仓库的基础上不断的把新的机制加入到你自己的 Loop 里。

那么说了这么多，怎么开始？非常简单，以第 1 节我们给出的 Prompt 为基础，修改一下，将其交给你的 Hermes、Codex、Qoder 等 Agent，他们会帮你设定 Loop，这几乎就是他们的本职工作。至于他怎么实现的，如果你没有兴趣可以不管，如果你有兴趣，可以让这些 Agent 讲给你听。

2.7 小结

Loop Engineering 的核心要点：

- **核心理念：** 从“手动 prompt Agent”升级为“设计自动运行的循环系统”
- **六个积木：** 调度、技能、隔离工作区、连接器、子智能体、记忆/状态
- **三个核心文件：** `LOOP.md`（岗位职责说明书）、`STATE.md`（工作日志）、`gate.yaml`（禁区地图）
- **三阶段信任：** L1 只看不动 → L2 自动低风险 → L3 高自主
- **配套工具：** 八个独立 npm CLI 工具，覆盖初始化、审计、成本估算、安全门禁等

- **与 Goal Engineering 的关系：** Loop 做持续维护，Goal 做一次性任务，两者互补
- **与 Shell Engineering 的关系：** Shell 搭框架，Loop 让框架自己转起来

值得强调的是，Loop Engineering 不只是一项革新，更是一种**思维方式的转变**。正如 Addy Osmani 在博客结尾说的：

“两个人在同一个 Loop 上可能得到完全相反的结果——一个人用它加速自己深入理解的工作，另一个人用它逃避理解工作。Loop 不知道区别，但你知道。”

设计 Loop 的难度不在于写代码，而在于你是否真的愿意——也有能力——从“操作员”变成“监督员”。

2.8 ■ 学点英语

中文	English	音标	说明
积木	Primitive	/ˈprɪmɪtɪv/	Loop 体系的基本构件，六个积木可任意组合搭配
分类	Triage	/triːˈɑːʒ/	Loop 启动后第一步——分析情况，判断优先级
子智能体	Sub-agent	/sʌb ˈeɪdʒənt/	在主 Agent 下独立运行的子任务执行者
验证者	Verifier	/ˈverɪfaɪər/	Maker-Checker 中负责检查的独立 Agent
安检	Gate	/geɪt/	程序硬性执行的机械安全检查，对照 gate.yaml
审计	Audit	/ˈɔːdɪt/	对 Loop 执行过程和产出进行审查评分
预算	Budget	/ˈbʌdʒɪt/	Loop 每次运行的 token 消耗上限
队列	Queue	/kjuː/	按顺序排队等待处理的任务列表